



**ANJALAI AMMAL - MAHALINGAM ENGINEERING
COLLEGE**

PROJECT TITLE :

**WhatNext Vision Motors : Shaping the Future of Mobility with
Innovation and Excellence.**

Team ID : SWTID-2026-7267

TEAM LEADER - S. LAKSHANA - lakshanaselvakumar251@gmail.com.

TEAM MEMBER - R.NIVEESH - niveesh777@gmail.com.

TEAM MEMBER - R.BHAVESH - rbhaves2006@gmail.com.

TEAM MEMBER - R.GEETHANJALI - narayanirameswaran@gmail.com.

TEAM MEMBER - S.HARINI - senthilvelvel378@gmail.com.

User Scenario:

The WhatsNext Vision Motors Salesforce system supports the customer vehicle ordering process through automated dealer selection, stock validation, and order status management.

- 1. The customer enters the required address details while placing a vehicle order.**
- 2. The system identifies and suggests the nearest available dealer based on the customer's address.**
- 3. The system checks the vehicle's stock availability before allowing the order to be created.**
- 4. Orders for out-of-stock vehicles are restricted or marked as Pending.**
- 5. Orders for vehicles available in stock are marked as Confirmed.**
- 6. A scheduled process automatically updates the status of bulk orders based on current stock availability.**

This process ensures accurate order handling, reduces manual work, and provides customers with clear order status information.

PROJECT OVERVIEW :

WhatsNext Vision Motors uses Salesforce to manage vehicles, dealers, customers, orders, test drives, and service requests. The project focuses on stock validation, dealer assignment, test-drive reminders, and automated order processing.

PROJECT OBJECTIVES :

- Manage vehicle, dealer, customer, order, test-drive, and service information.**

- Prevent orders for vehicles that are out of stock.
- Assign dealers based on customer location.
- Send automated test-drive reminders.
- Process pending orders based on vehicle stock availability.
- Automate repetitive business operations using Flow, Apex, Batch Apex, and Scheduled Apex.

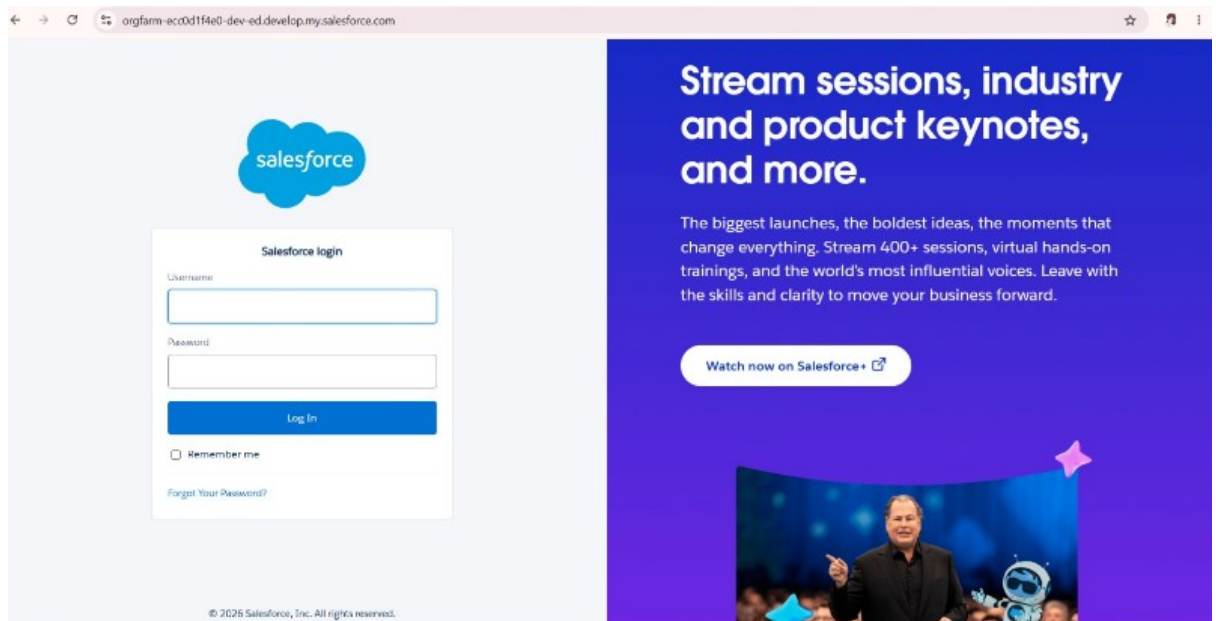
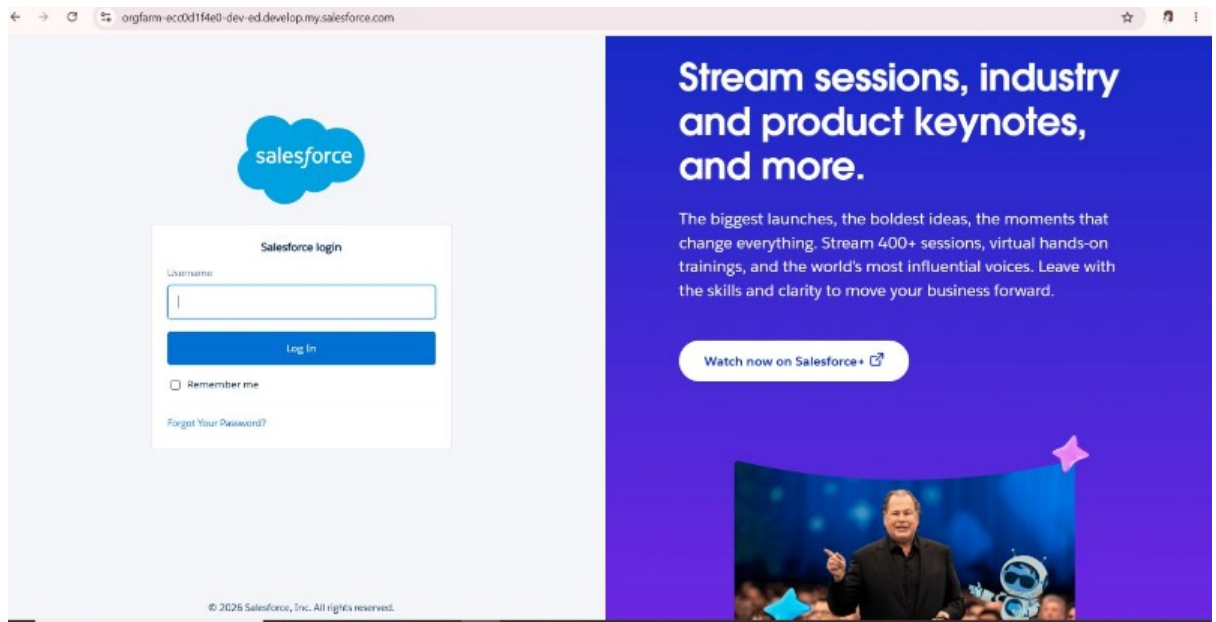
LEARNING OUTCOMES :

- Data Modelling
- Fields and Relationships
- Lightning App Builder
- Record-Triggered Flows
- Apex and Apex Triggers
- Batch Apex
- Scheduled Apex

MILESTONE 1 – DEVELOPER ACCOUNT :

Activity 1: Create Developer Account :

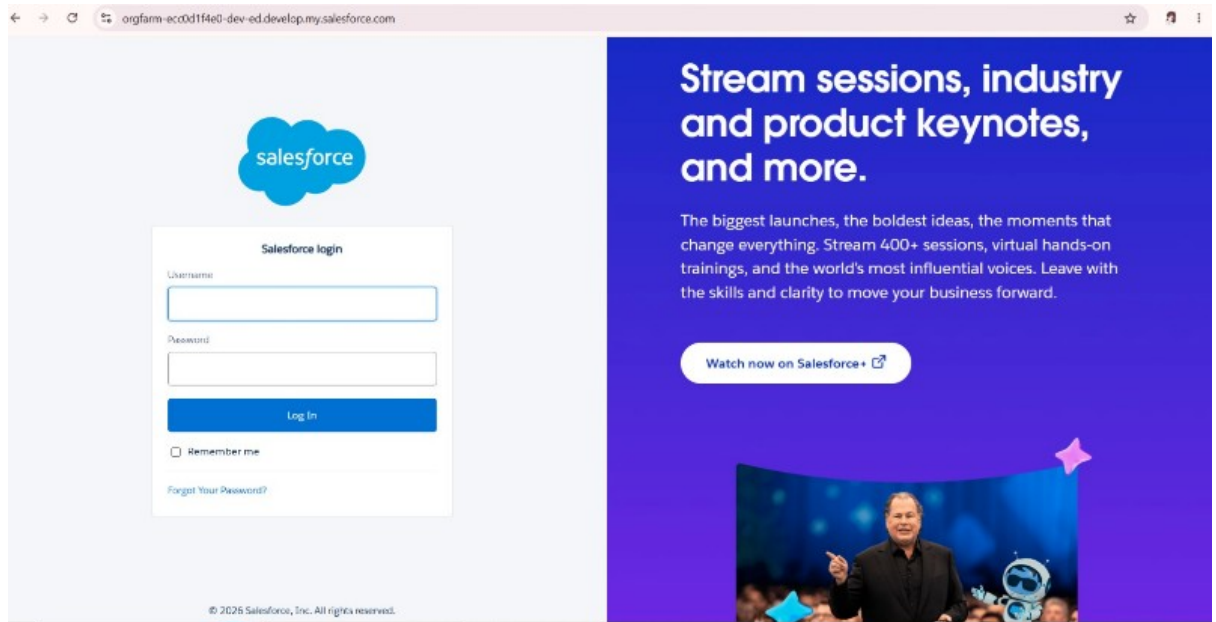
1. Open the Salesforce Developer signup page.
2. Enter the required personal and organization details.
3. Select Developer as the role.
4. Enter India as the country and provide the postal code.
5. Enter a unique username in the required format.
6. Click Sign Me Up.



Activity 2: Activate Account :

1. Open the registered email inbox.
2. Open the Salesforce verification email.
3. Click Verify Account.
4. Set the account password and security question.

5. Click Change Password.
6. Access the Salesforce Setup page.



MILESTONE 2 – OBJECTS AND RELATIONSHIPS :

Required Custom Objects :

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

Activity 1: Create Vehicle Object :

1. Open Setup → Object Manager → Create → Custom Object.



1. Enter the label name : Vehicle

SETUP
New Custom Object

Custom Object Definition Edit Save Save & New Cancel

Custom Object Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Account
 Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object in the API.

Object Name Example: Account
 Description

Context Sensitive Help Setting
☒ Open the standard Salesforce.com Help & Training window
☐ Open a window using a Visualforce page

Content Name

Enter Record Name Label and Format

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field

Record Name Example: Account Name
 Data Type Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

2. Enter Vehicle as the label.
3. Enter Vehicles as the plural label.

SETUP
New Custom Object

Custom Object Definition Edit Save Save & New Cancel

Custom Object Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Account
 Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object in the API.

Object Name Example: Account
 Description

Context Sensitive Help Setting
☒ Open the standard Salesforce.com Help & Training window
☐ Open a window using a Visualforce page

Content Name

Enter Record Name Label and Format

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field

Record Name Example: Account Name
 Data Type Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

4. Set Vehicle Name as the record name.
5. Select Text as the data type.

6. Enable Allow Reports and Allow Search.
7. Click Save.

Optional Features

- ☒ Allow Reports
- ☐ Allow Activities
- ☐ Track Field History
- ☐ Allow in Chatter Groups
- ☐ Enable Licensing

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more](#)

- ☒ Allow Sharing
- ☒ Allow Bulk API Access
- ☒ Allow Streaming API Access

Deployment Status

☐ In Development

☒ Deployed

Search Status

When this setting is enabled, your users can find records of this object type when they search. [Learn more](#)

☒ Allow Search

Object Creation Options (Available only when custom object is first created)

- ☐ Add Notes and Attachments related list to default page layout
- ☐ Launch New Custom Tab Wizard after saving this custom object

[Save](#) [Save & Close](#) [Cancel](#)

Activity 2: Create Vehicle Dealer Object :

1. Create a new custom object from Object Manager.

Optional Features

- ☒ Allow Reports
- ☐ Allow Activities
- ☐ Track Field History
- ☐ Allow in Chatter Groups
- ☐ Enable Licensing

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more](#)

- ☒ Allow Sharing
- ☒ Allow Bulk API Access
- ☒ Allow Streaming API Access

Deployment Status

☐ In Development

☒ Deployed

Search Status

When this setting is enabled, your users can find records of this object type when they search. [Learn more](#)

☒ Allow Search

Object Creation Options (Available only when custom object is first created)

- ☐ Add Notes and Attachments related list to default page layout
- ☐ Launch New Custom Tab Wizard after saving this custom object

[Save](#) [Save & Close](#) [Cancel](#)

2. Enter Vehicle Dealer as the label.
3. Enter Vehicle Dealers as the plural label.
4. Set Dealer Name as the record name.
5. Select Text as the data type.
6. Enable Allow Reports and Allow Search.
7. Click Save.

Activity 3: Create Remaining Objects :

Create the following objects using the same procedure:

- Vehicle Customer
- Vehicle Order
- Vehicle Test Drive
- Vehicle Service Request

MILESTONE 3 – CUSTOM TABS :

Activity 1: Create Vehicle Tab :

1. Open Setup → Tabs.
2. Select New under Custom Object Tabs.
3. Select the Vehicle object.

The screenshot displays the Salesforce Setup interface for creating a custom object. The left sidebar shows the 'Setup' menu with 'Object Manager' selected. The main content area is titled 'Vehicle' and contains the 'Custom Object Information' section. This section includes fields for 'Label' (Vehicle), 'Plural Label' (Vehicles), 'Object Name' (Vehicle), and 'Description'. It also features a 'Context Sensitive Help Setting' with options to 'Open the standard Salesforce.com Help & Training window' or 'Open a webpage using a Visualforce page'. At the bottom, the 'Enter Record Name Label and Format' section shows the 'Record Name' field set to 'Vehicle Name'.

Setup | Home | Object Manager

SEARCH SETUP

SETUP > OBJECT MANAGER

Vehicle

Details

- Fields & Relationships
- Page Layouts
- Lightning Record Pages
- Buttons, Links, and Actions
- Compact Layouts
- Field Sets
- Object Limits
- Record Types
- Related Lookup Filters
- Search Layouts
- List View Button Layout
- Restriction Rules

Custom Object Information

The singular and plural labels are used in tabs, page layouts, and reports. Be careful when changing the name or label as it may affect existing integrations and merge templates.

Label: Example: Account

Plural Label: Example: Accounts

Starts with vowel sound: ☐

The Object Name is used when referencing the object via the API.

Object Name: Example: Account

Description:

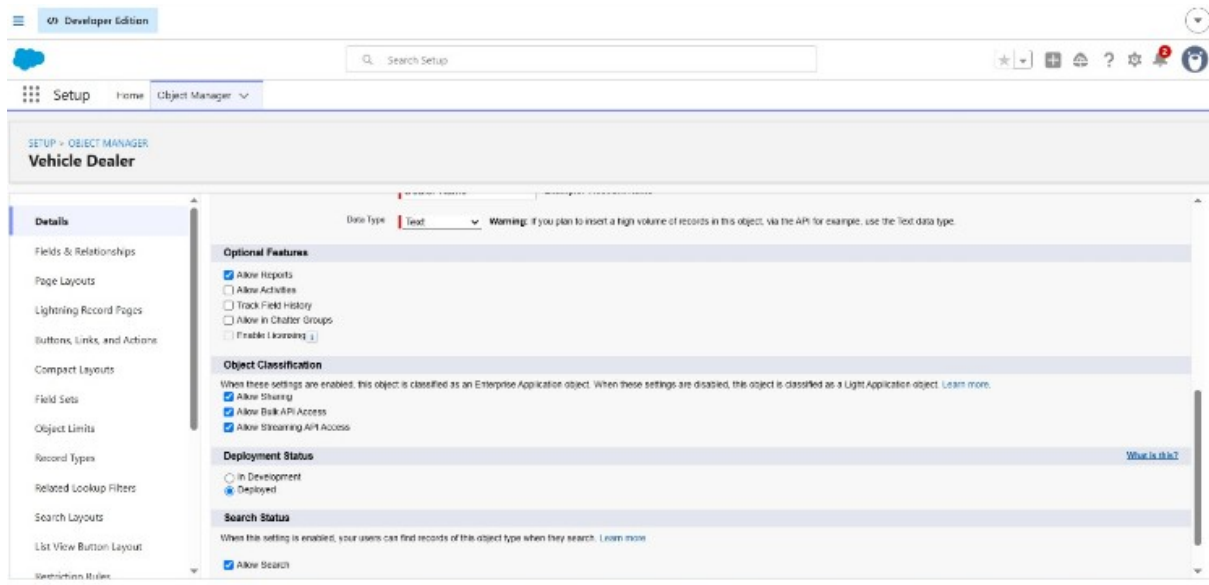
Context Sensitive Help Setting: ☒ Open the standard Salesforce.com Help & Training window
☐ Open a webpage using a Visualforce page

Context Name:

Enter Record Name Label and Format

The Record Name appears in page layouts, key tabs, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field is always called "name" when referenced via the API.

Record Name: Example: Account Name



4. Select a suitable tab style.
5. Keep profile and app settings as default.
6. Click Save.

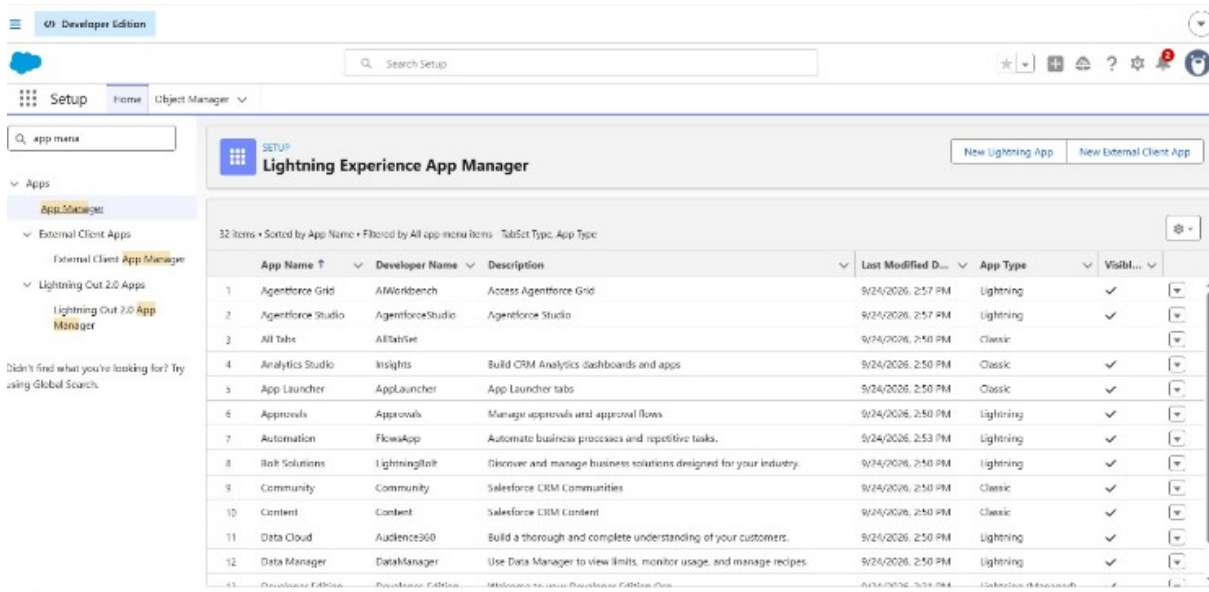
Activity 2: Create Remaining Tabs :

Create custom tabs for Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, and Vehicle Service Request.

MILESTONE 4 – LIGHTNING APP :

Activity 1: Create Lightning App :

1. Open Setup → App Manager.
2. Click New Lightning App.



3. Enter the app name as WhatNext Vision Motors.
4. Enter a suitable description.
5. Keep branding and utility settings as required.

6. Add the required navigation items:

- Vehicle
- Vehicle Dealer
- Vehicle Customer
- Vehicle Order
- Vehicle Test Drive
- Vehicle Service Request
- Reports
- Dashboard

7. Add the required user profile.

8. Click Save & Finish.

MILESTONE 5 – FIELDS AND RELATIONSHIPS :

Vehicle Fields :

- Vehicle Name – Text
- Vehicle Model – Picklist
- Stock Quantity – Number
- Price – Currency
- Dealer – Lookup Relationship
- Status – Picklist

Vehicle Dealer Fields :

- Dealer Name – Text
- Dealer Location – Text
- Dealer Code – Auto Number
- Phone – Phone
- Email – Email

Vehicle Order Fields :

- Vehicle Customer – Lookup
- Vehicle – Lookup
- Order Date – Date
- Status – Picklist

Vehicle Customer Fields :

- Customer Name – Text
- Email – Email
- Phone – Phone
- Address – Text
- Preferred Vehicle Type – Picklist

Vehicle Test Drive Fields :

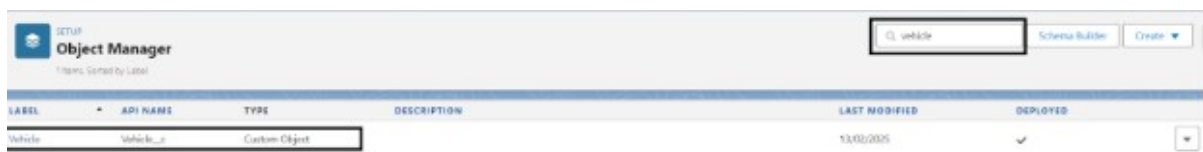
- Vehicle Customer – Lookup
- Vehicle – Lookup
- Test Drive Date – Date
- Status – Picklist

Vehicle Service Request Fields :

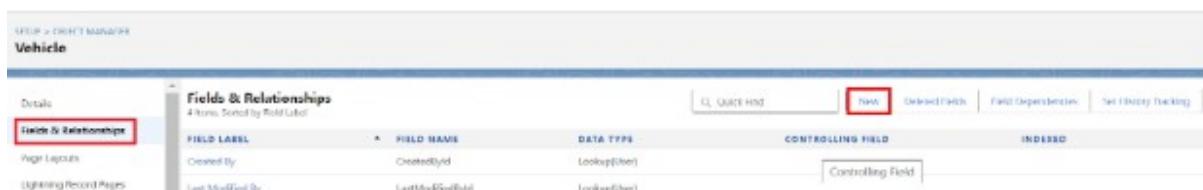
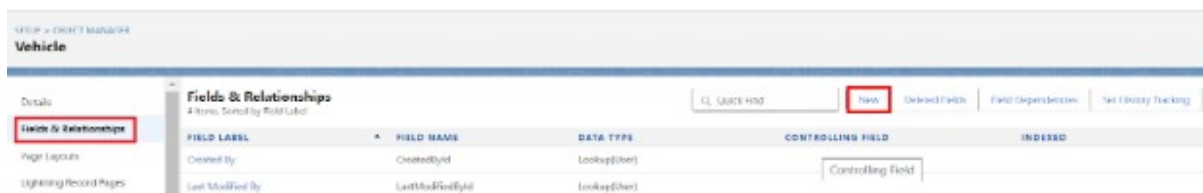
- Vehicle Customer – Lookup
- Vehicle – Lookup
- Service Date – Date
- Issue Description – Text
- Status – Picklist

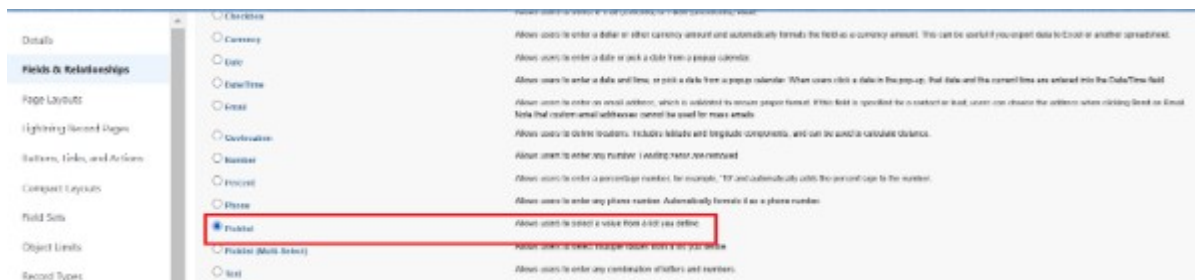
Activity 1: Create Vehicle Model Field

1. Open Setup → Object Manager → Vehicle → Fields & Relationships.



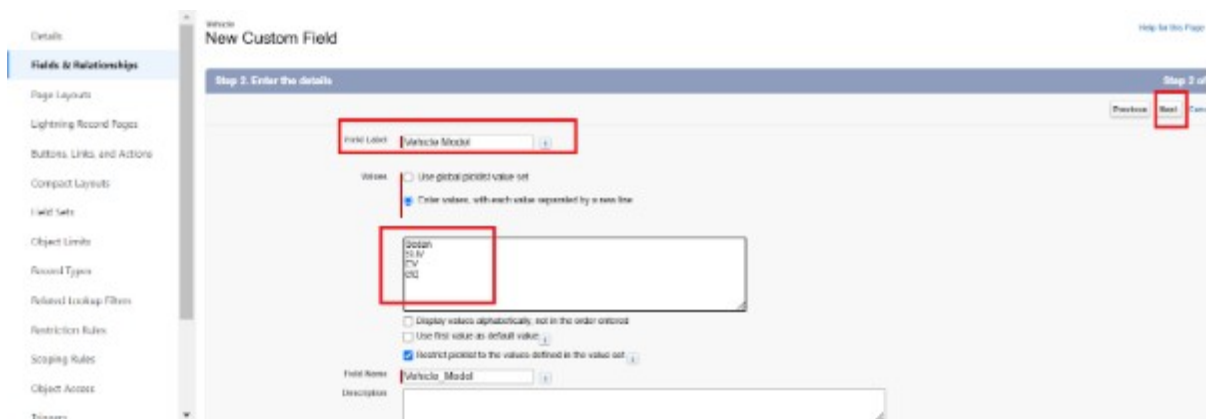
2. Click New.
3. Select Picklist.





4. Enter Vehicle Model as the field label.

5. Add the required vehicle types.



6. Click Next → Next → Save.

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

Vehicle_Model__c (Picklist: Sedan, SUV, EV, etc.)
Stock_Quantity__c (Number)
Price__c (Currency)
Dealer__c (Lookup to Dealer__c)
Status__c (Picklist: Available, Out of Stock, Discontinued)
Vehicle_Dealer__c (Custom Object)
Dealer_Name__c (Text)
Dealer_Location__c (Text)
Dealer_Code__c (Auto Number)
Phone__c (Phone)
Email__c (Email)
Vehicle_Order__c (Custom Object)
Customer__c (Lookup to Customer__c)
Vehicle__c (Lookup to Vehicle__c)
Order_Date__c (Date)
Status__c (Picklist: Pending, Confirmed, Delivered, Canceled)
Vehicle_Customer__c (Custom Object)
Customer_Name__c (Text)
Email__c (Email)
Phone__c (Phone)
Address__c (Text)
Preferred_Vehicle_Type__c (Picklist: Sedan, SUV, EV, etc.)

Activity 2: Create Stock Quantity Field :

1. Click New under Fields & Relationships.
2. Select Number.



3. Enter Stock Quantity as the field label.

4. Click Next → Next → Save.

Activity 3: Create Dealer Lookup :

1. Click New.
2. Select Lookup Relationship.

Vehicle

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Restriction Rules

Scoping Rules

Vehicle

New Custom Field

Step 1: Choose the field type

Specify the type of information that the custom field will contain.

Data Type

☐ Name (Text)

☐ Auto Number

☐ Formula

☐ Roll-Up Summary

☒ Lookup Relationship

☐ Master-Detail Relationship

Next

Vehicle

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Vehicle

New Relationship

Step 3: Enter the label and name for the lookup field

Field Label: Dealer

Field Name: Dealer

Next

3. Select Vehicle Dealer as the related object.

Vehicle

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Vehicle

New Relationship

Step 4: Establish field-level security for reference field

Field Label: Dealer

Field Name: Dealer

Field Type: Lookup

Description:

Select the profiles to which you want to grant edit access to this field via field-level security. The field will be hidden from all profiles if you do not add it to field-level security.

Field-Level Security for Profile

Analytics Cloud Integration User

Analytics Cloud Security User

Vehicle

Next

4. Complete the field settings.
5. Click Save.

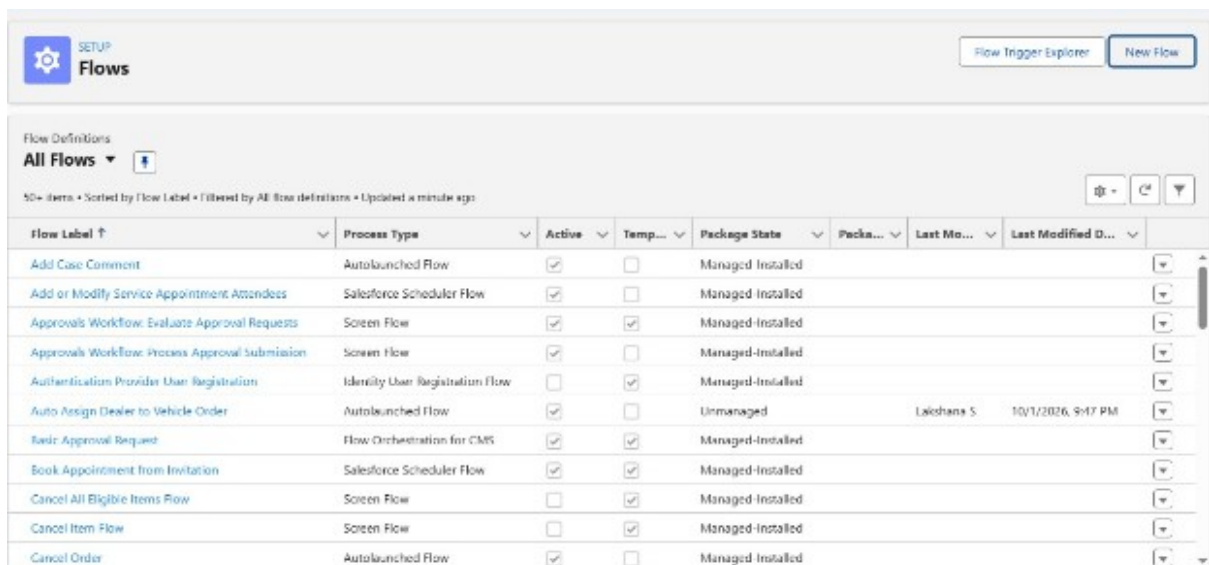
Activity 4: Create Remaining Fields :

Create the remaining fields and relationships according to the project data model.

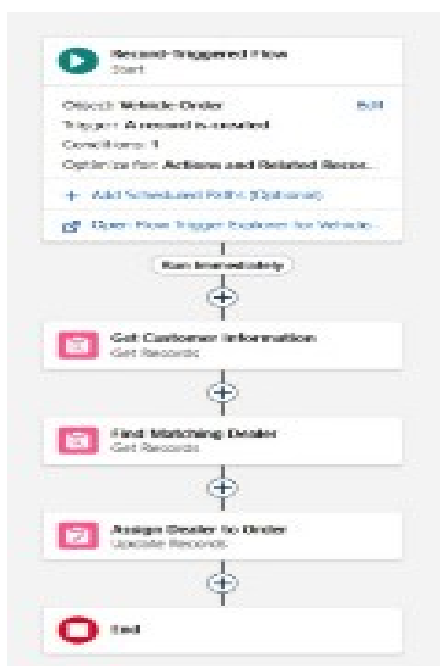
MILESTONE 6 – FLOW AUTOMATION :

Flow 1: Automatic Dealer Assignment :

1. Open Setup → Flows → New Flow.



Flow Label ↑	Process Type	Active	Temp...	Package State	Packa...	Last Mo...	Last Modified D...
Add Case Comment	Autolaunched Flow	☒	☐	Managed-Installed			
Add or Modify Service Appointment Attendees	Salesforce Scheduler Flow	☒	☐	Managed-Installed			
Approvals Workflow: Evaluate Approval Requests	Screen Flow	☒	☒	Managed-Installed			
Approvals Workflow: Process Approval Submission	Screen Flow	☒	☐	Managed-Installed			
Authentication Provider User Registration	Identity User Registration Flow	☐	☒	Managed-Installed			
Auto Assign Dealer to Vehicle Order	Autolaunched Flow	☒	☐	Unmanaged		Laishana S	10/1/2026, 9:47 PM
Book Approval Request	Flow Orchestration for CMS	☒	☒	Managed-Installed			
Book Appointment from Invitation	Salesforce Scheduler Flow	☒	☒	Managed-Installed			
Cancel All Eligible Items Flow	Screen Flow	☐	☒	Managed-Installed			
Cancel Item Flow	Screen Flow	☐	☒	Managed-Installed			
Cancel Order	Autolaunched Flow	☒	☐	Managed-Installed			



Select Record-Triggered Flow.

2. Select Vehicle Order as the object.
3. Set the trigger to record creation.
4. Set the entry condition as Status = Pending.
5. Add Get Records to retrieve customer information.

 Get Records

Label

Get Customer Information.

API Name

Get_Customer_Information

Description

Get Records of This Object

Object

Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements

All Conditions Are Met (AND) ▼

Field	Operator	Value
Id	Equals ▼	Aa \$Record > Vehicle Customer ×

+ Add Condition

6. Retrieve the customer using the Vehicle Customer lookup.
7. Add another Get Records element to retrieve the relevant dealer.
8. Use customer location and dealer location for the matching condition.
9. Add Update Records to assign the dealer to the order.

Update Records

* Label	* API Name 
Assign Dealer to Order	Assign_Dealer_to_Order
Description	

* How to Find Records to Update and Set Their Values

- ☐ Use the vehicle order record that triggered the flow
- ☐ Update records related to the vehicle order record that triggered the flow
- ☒ Use the IDs and all field values from a record or record collection
- ☐ Specify conditions to identify records, and set fields individually

Select Record(s) to Update

Record or Record Collection
 Vehicle Dealer from Get Nearest Dealer X

Make sure that each record has an ID. Otherwise the flow can't find the records to update, and it fails. 

Save as

Save As	
A New Version	
* Flow Label	* Flow API Name 
Auto Assign Dealer	Auto_Assign_Dealer
Description	

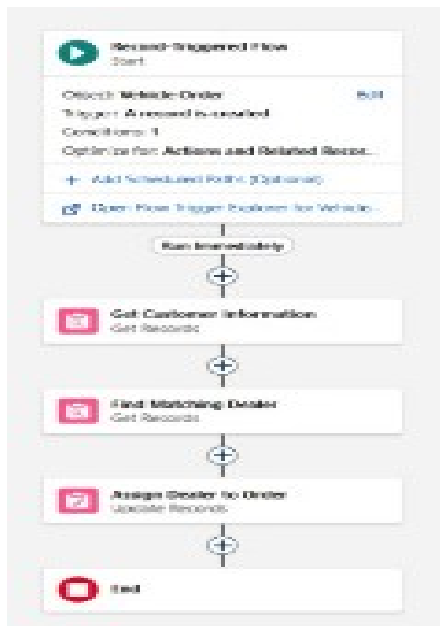
[Show Advanced](#)

Cancel

Save

10. Save the flow as Auto Assign Dealer.

11. Activate the flow.



Flow 2: Test Drive Reminder :

1. Create a new Record-Triggered Flow.
2. Select Vehicle Test Drive.

Save as

Save As

A New Version

Flow Label

Test Drive Reminder

* Flow API Name ⓘ

Test_Drive_Reminder

Description

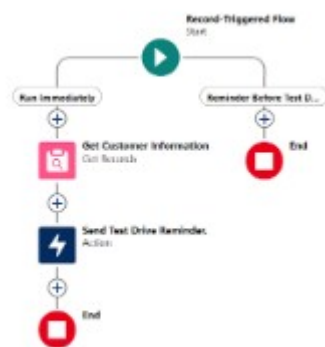
Show Advanced

Cancel Save

Step 7 : Activate Flow

n Debug View Tests Save As New Version Save Activate

3. Configure the flow for record creation or update.
4. Set the condition as Status = Scheduled.
5. Add a scheduled path using Test Drive Date.
6. Set the scheduled path for 1 day before the test drive.
7. Retrieve customer information using Get Records.
8. Add a Send Email action.
9. Configure the customer email address as the recipient.
10. Use the subject Reminder: Your Test Drive is Tomorrow!
11. Save the flow as Test Drive Reminder.
12. Activate the flow.



Send Email

Recipient Address List (Included) ☒

Recipient ID (Not Included) ☐

Related Record ID (Not Included) ☐

Rich-Text-Formatted Body (Included) ☒

Sender Email Address (Not Included) ☐

Sender Type (Not Included) ☐

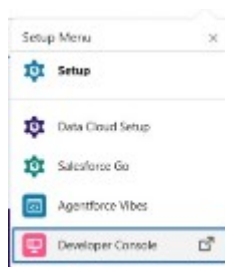
Subject (Included) ☒

Use Line Breaks (Included) ☒

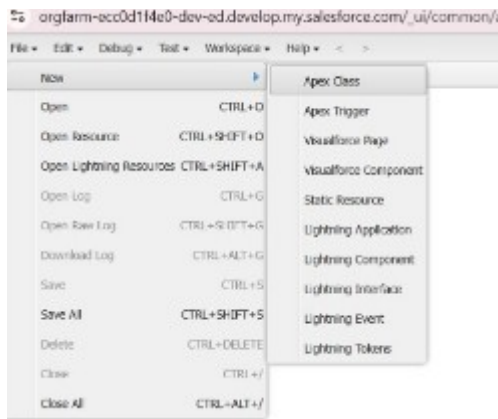
MILESTONE 7 – APEX, TRIGGER AND BATCH PROCESSING :

Activity 1: Create Apex Handler :

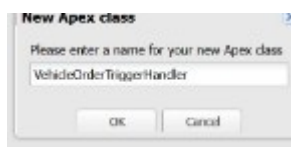
1. Open Developer Console.



2. Select File → New → Apex Class.

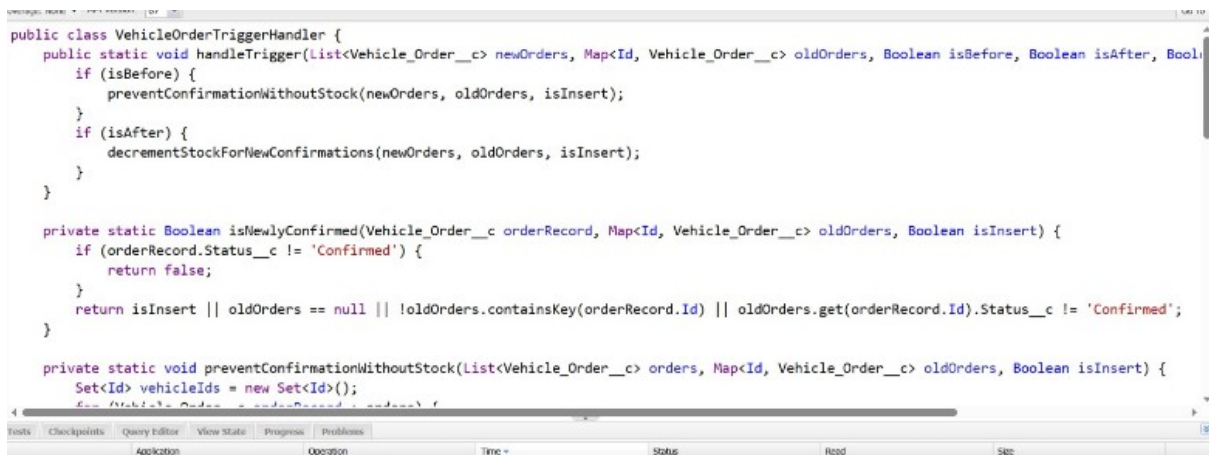


3. Enter the class name as VehicleOrderTriggerHandler.



4. Implement stock validation and stock update logic.

5. Use bulk-safe processing for multiple order records.



Source code :

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean
isUpdate) {
```

```

    if (isBefore) {
        if (isInsert || isUpdate) {
            preventOrderIfOutOfStock(newOrders);
        }
    }

    if (isAfter) {
        if (isInsert || isUpdate) {
            updateStockOnOrderPlacement(newOrders);
        }
    }
}

// Method to prevent orders when the vehicle is out of stock
private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null) {
            vehicleIds.add(order.Vehicle__c);
        }
    }
    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE
Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }
        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c <= 0) {
                    order.addError('This vehicle is out of stock. Order cannot be placed.');
```



```

    }
}

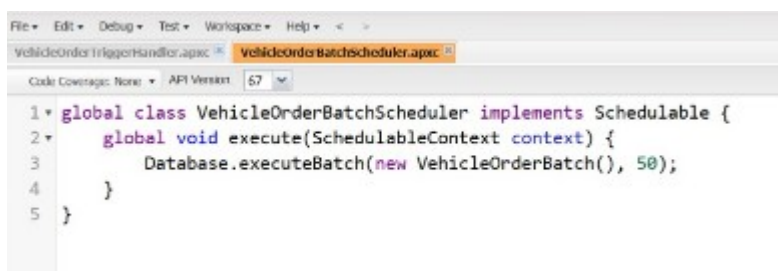
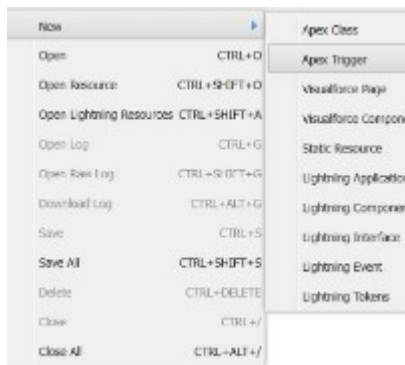
// Method to update vehicle stock when an order is placed
private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
    }
    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE
Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }
        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c > 0) {
                    vehicle.Stock_Quantity__c -= 1;
                    vehiclesToUpdate.add(vehicle);
                }
            }
        }
        if (!vehiclesToUpdate.isEmpty()) {
            update vehiclesToUpdate;
        }
    }
}
}

```

6. Save the Apex class.

Activity 2: Create Trigger :

1. Create a new Apex Trigger.



2. Select Vehicle Order as the object.

3. Configure the trigger for required insert and update events.

4. Call the VehicleOrderTriggerHandler class.

Source code :

trigger VehicleOrderTrigger on Vehicle_Order__c (before insert,before update, after insert, after update) {

VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore,Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
}

5. Save the trigger.

Activity 3: Stock Validation :

1. Retrieve vehicle records related to the orders.
2. Check the available stock quantity.
3. Display an error when stock quantity is zero or below.
4. Prevent the order from being created when the vehicle is unavailable.

Activity 4: Update Vehicle Stock :

1. Identify confirmed orders.
2. Retrieve the related vehicle records.
3. Reduce the stock quantity after confirmed order placement.
4. Update the vehicle records.

Activity 5: Create Batch Apex :

1. Create a Batch Apex class named VehicleOrderBatch.



2. Retrieve pending orders.
3. Check the current vehicle stock.
4. Update the order status based on stock availability.

```

1 * global class VehicleOrderBatch implements Database.Batchable<Object> {
2
3     * global Database.QueryLocator start(Database.BatchableContext bc) {
4     *     return Database.getQueryLocator([
5     *         SELECT Id, Status__c, Vehicle__c
6     *         FROM Vehicle_Order__c
7     *         WHERE Status__c = 'Pending'
8     *     ]);
9     }
10
11 * global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderlist) {
12     Set<Id> vehicleIds = new Set<Id>();
13
14     * for (Vehicle_Order__c order : orderlist) {
15     *     if (order.Vehicle__c != null) {
16     *         vehicleIds.add(order.Vehicle__c);
17     *     }
18     }
19
20     * if (vehicleIds.isEmpty()) {

```

Set the status as Confirmed when stock is available.

Source code :

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

global Database.QueryLocator start (Database.BatchableContext bc) {

return Database.getQueryLocator([
SELECT Id, Status__c, Vehicle__c
FROM Vehicle_Order__c
WHERE Status__c
'Pending'
]);
}

global void execute(Database.BatchableContext bc, List<Vehicle_Order_c> orderList) {
Set<Id> vehicleIds = new Set<Id>();
for (Vehicle_Order_c order orderList) {
if (order.Vehicle_c != null) {
vehicleIds.add(order.Vehicle__c);
}
}
if (!vehicleIds.isEmpty()) {
Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
for (Vehicle_c vehicle: [SELECT Id, Stock Quantity_c FROM Vehicle_c WHERE Id IN
vehicleIds]) {
vehicleStockMap.put(vehicle. Id, vehicle);
}
List<Vehicle_Order_c> ordersToUpdate = new List<Vehicle_Order_c>(); List<Vehicle__c>
vehiclesToUpdate = new List<Vehicle__c>();
for (Vehicle_Order_c order orderList) {
if (vehicleStockMap.containsKey(order. Vehicle__c)) {
Vehicle_c vehicle = vehicleStockMap.get(order. Vehicle__c);
if (vehicle.Stock_Quantity__c > 0) {
order.Status__c = 'Confirmed';
```

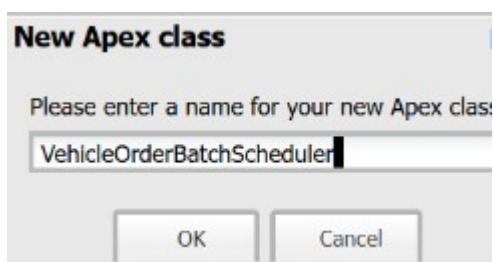
```

vehicle.Stock_Quantity__c -= 1;
ordersToUpdate.add(order);
vehiclesToUpdate.add(vehicle);
}
}
}
if (!ordersToUpdate.isEmpty()) {
    update ordersToUpdate;
}
if (!vehiclesToUpdate.isEmpty()) {
    update vehiclesToUpdate;
}
}
global void finish(Database.BatchableContext bc) {
    System.debug('Vehicle order batch job completed.');
```

5. Keep the status as Pending when stock is unavailable.
6. Save the Batch Apex class.

Activity 6: Create Scheduled Apex :

1. Create the VehicleOrderBatchScheduler class.



2. Implement the Schedulable interface.

```
1+ global class VehicleOrderBatchScheduler implements Schedulable {  
2+     global void execute(SchedulableContext sc) {  
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();  
4         Database.executeBatch(batchJob, 50);  
5     }  
6 }  
7
```

Source code :

```
global class VehicleOrderBatchScheduler implements Schedulable {  
    global void execute(SchedulableContext sc) {  
        VehicleOrderBatch batchJob = new VehicleOrderBatch();  
        Database.executeBatch(batchJob, 50); // 50 is the batch size  
    }  
}
```

3. Execute the Vehicle Order Batch class from the scheduler.
4. Set the required batch size.
5. Save the scheduler class.

Activity 7: Schedule Batch Processing :

1. Open Execute Anonymous Window.
2. Define the required Cron expression.
3. Schedule the VehicleOrderBatchScheduler class.
4. Verify the scheduled job from Setup → Scheduled Jobs.

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

Source code :

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
```

```
System.schedule('Daily Vehicle Order Processing', cronExp, new
VehicleOrderBatchScheduler());
```



Name	Job Name	Frequency	Next Run	Status	Owner	Created By	Created Date	Last Run Date	Last Run Status	Last Run Error
Daily Vehicle Order Processing	Daily Vehicle Order Processing	0 0 12 * * ?	12/15/2023 12:00 PM	Active	System Administrator	System Administrator	12/15/2023 12:00 PM	12/15/2023 12:00 PM	Success	
Program Status Update Cron Job	Program Status Update Cron Job	0 0 12 * * ?	12/15/2023 12:00 PM	Active	System Administrator	System Administrator	12/15/2023 12:00 PM	12/15/2023 12:00 PM	Success	

EXPECTED PROCESS :

Customer → Vehicle Selection → Stock Validation → Order Creation → Dealer Assignment → Order Status Update → Stock Update → Scheduled Batch Processing

PROJECT COMPLETION :

The completed Salesforce implementation provides centralized vehicle and customer management, automated dealer assignment, stock validation, test-drive reminders, Apex-based business rules, and scheduled order processing.